



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 05 – Lecture 08: Regression Foundations and Least Squares

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Topic focus: **for this one model, the minimisation has an answer you can write down**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

Quick Links

[What regression is](#)

[Models](#)

[Steps](#)

[Simple linear regression](#)

[The derivation](#)

[Five points](#)

[Line of best fit](#)

[Matrix form](#)

[Two roads](#)

[Cautions](#)

[Summary](#)

Agenda

What regression is

Regression examples and models

Steps in a regression analysis

Simple linear regression

Least-squares estimation: the derivation

The five-point calculation

The line of best fit and its properties

The normal equations in matrix form

Two roads to the same answer

What least squares will not tell you

Summary

How this lecture works

Every idea appears twice

1. **By hand** — five points, $x = 1, 2, 3, 4, 5$. Every intermediate quantity is an integer or a clean decimal, so the whole fit goes on paper.
2. **In code** — `np.polyfit`, `LinearRegression` and $(X^T X)^{-1} X^T y$ on the same five numbers, then on 442 real patients.

Commit before you look

Six slides ask you to predict a number before it appears. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is **a derivation**: two partial derivatives set to zero, solved, giving $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1 \bar{x}$. It is examinable, and we do it on the board.

Where we are

Unit II said *what* to minimise

The mean squared error

$J(w) = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$. A number that says how wrong a model is.

Unit III said *how* to minimise it

Gradient descent, and its stochastic, mini-batch, momentum and adaptive relatives. Take a step downhill; repeat.

Unit IV made the table fit to use

Cleaned, scaled, encoded, reduced, split.

Unit V asks a new question

Take the simplest possible model — a straight line — and the same squared-error objective. **Do we actually need to iterate?**

The answer, for today only

No. Setting two derivatives to zero gives two linear equations in two unknowns, and **you solve them once**. We will then run gradient descent on the same data and watch it crawl to exactly the number the formula produced in one line.

One seed, and every number in this deck reproduces

The data

Bundled `load_diabetes`, `load_breast_cancer` and `load_wine`, the published Anscombe table, and the five-point revision example. Nothing is downloaded, and none of it is randomly generated.

The randomness

One seed for the whole lecture:
`np.random.default_rng(0)`, and
`random_state=0` on every split and every stochastic estimator. Each listing that draws or shuffles shows it.

Why this matters to you

Every number in this deck is printed by one script. Run the code as written and you get the same numbers — which is the only reason to believe a number on a slide.

Regression: predicting a number

Modelling how a **quantitative** response varies with one or more predictors.

The vocabulary, which the exam uses

y response, dependent, target

x predictor, regressor, feature

ε error, disturbance, noise

The model splits y into a **systematic** part $f(x)$ and a **random** part ε :

$$y = f(x) + \varepsilon.$$

Regression against classification

y continuous $\Rightarrow$ regression; y a label $\Rightarrow$ classification. Rupees, marks, temperature, disease progression are regression. Spam / not spam is not.

And the word itself

Galton, 1886: tall fathers have sons who are tall but *less* so — “regression towards mediocrity”. The name stuck to the method, not the phenomenon.

Two jobs a regression can do

Prediction

Given a new x , what is y ? Nobody has to understand the model. Judge it by held-out error.

What will this flat sell for?

Explanation

Which predictors matter, in which direction, by how much? Judge it by whether the coefficients can be trusted.

Does an extra hour of revision raise the mark, and by how much?

The two jobs pull in different directions

A model can predict well with coefficients that mean nothing — Lecture 6 showed exactly that with the dummy variable trap, where two completely different coefficient vectors gave *identical* predictions. **Decide which job you are doing before you choose the model.**

Today's model does both, which is why it is still the first one taught.

Where regression is actually used

Field	Response y	Predictors x
Medicine	disease progression in a year	BMI, blood pressure, six serum measures
Real estate	sale price, in rupees	area, rooms, age, locality
Energy	tomorrow's peak load	temperature, weekday, last week's load
Agriculture	yield per hectare	rainfall, fertiliser, soil nitrogen
Manufacturing	tensile strength	alloy fractions, furnace temperature
Finance	next-day return	yesterday's return, volume, volatility
Education	end-semester mark	attendance, internal marks, hours studied

The one we will use all lecture

`load_diabetes`: 442 patients, 10 measurements, and a quantitative score of disease progression one year after baseline. It ships with scikit-learn, so **nothing downloads**.

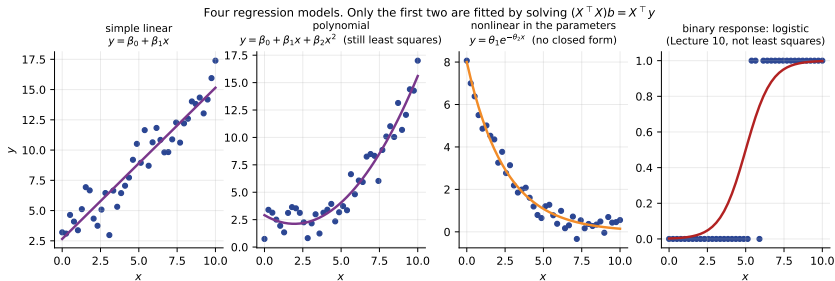
The family of regression models

Model	Form	Fitted by
Simple linear	$y = \beta_0 + \beta_1 x + \varepsilon$	least squares, closed form
Multiple linear	$y = \beta_0 + \sum_j \beta_j x_j + \varepsilon$	least squares, closed form (L11)
Polynomial	$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots$	least squares, closed form
Log / reciprocal terms	$y = \beta_0 + \beta_1 \log x + \varepsilon$	least squares, closed form
Ridge, lasso	least squares + a penalty	closed form / convex solver (L12)
Logistic	$\Pr(y = 1) = \sigma(\beta_0 + \beta_1 x)$	maximum likelihood, iterative (L10)
Nonlinear	$y = \theta_1 e^{-\theta_2 x} + \varepsilon$	numerical optimisation
Nonparametric	trees, k -NN, splines	no global equation

The dividing line is not curvature. It is the parameters

A model is **linear** when it is linear *in the parameters*. $y = \beta_0 + \beta_1 x + \beta_2 x^2$ draws a parabola and is a linear model. $y = \beta_0 + \beta_0 \beta_1 x$ draws a straight line and is *not*.

Drawn: four models, one fitting method between them



Panels 1 and 2 are both solved by $(X^T X)b = X^T y$; only the columns of X differ. Panel 3 has a parameter inside an exponential and needs an iterative solver. Panel 4 has a 0/1 response, so squared error is the wrong objective altogether — that is Lecture 10.

The same normal equations, three different designs

Fit degree 1, 2 and 3 to the *same* 40 points, each time by solving $(X^T X)b = X^T y$:

```
P = np.column_stack([xp**k for k in range(deg + 1)]) # the design matrix
bhat = np.linalg.solve(P.T @ P, P.T @ yp) # the normal equations
```

degree 1	design (40, 2)	coefficients [4.4626 -0.4485]	SSE 265.953
degree 2	design (40, 3)	coefficients [1.7027 -0.4485 0.8751]	SSE 22.655
degree 3	design (40, 4)	coefficients [1.7027 -0.4331 0.8751 -0.0027]	SSE 22.649

The same normal equations, three different designs

Fit degree 1, 2 and 3 to the *same* 40 points, each time by solving $(X^T X)b = X^T y$:

```
P = np.column_stack([xp**k for k in range(deg + 1)]) # the design matrix
bhat = np.linalg.solve(P.T @ P, P.T @ yp) # the normal equations
```

degree 1	design (40, 2)	coefficients [4.4626 -0.4485]	SSE 265.953
degree 2	design (40, 3)	coefficients [1.7027 -0.4485 0.8751]	SSE 22.655
degree 3	design (40, 4)	coefficients [1.7027 -0.4331 0.8751 -0.0027]	SSE 22.649

Data generated from $y = 2 - 0.5x + 0.8x^2 + \varepsilon$. **One solver, three models.** Everything in this lecture about the straight line transfers, unchanged, to any design matrix — which is why we spend the time on it.

The eight steps

1. **Statement of the problem.** What question, in words, before any data.
2. **Selection of variables.** What is y , what could be x , in what units.
3. **Data collection.** How many rows, how gathered, how split.
4. **Model specification.** Write the equation down.
5. **Choice of fitting method.** Least squares, maximum likelihood, penalised least squares.
6. **Fitting.** Compute the estimates.
7. **Model validation.** Residuals, held-out error, assumptions.
8. **Use.** Predict, or interpret — and say which.

Steps 1–3 are where analyses die

By the time you type `fit()` the important decisions are already made. A perfect fit to the wrong response answers the wrong question perfectly.

And step 7 sends you back

Validation is not a rubber stamp. If the residuals show structure you return to step 4, not to step 6. That loop is Lecture 9's subject.

All eight steps, once, on real patients

```
1 problem    : does body mass index predict disease progression?
2 variables  : y = progression score; x = bmi (centred and scaled here)
3 data       : 442 patients, 331 train / 111 test
4 model      :  $y = b_0 + b_1 x + e$ 
5 method     : least squares
6 fit        : LinearRegression  b0 = 153.2257  b1 = 1016.9235
               by hand           b0 = 153.2257  b1 = 1016.9235
7 check      : train RMSE 61.711   test RMSE 64.664
               training residuals sum to 2.274e-12
               5-fold RMSE 62.605 +/- 2.888
               always predicting ybar gives RMSE 77.006
8 use        : one sd more bmi predicts 49.4 more progression points
```

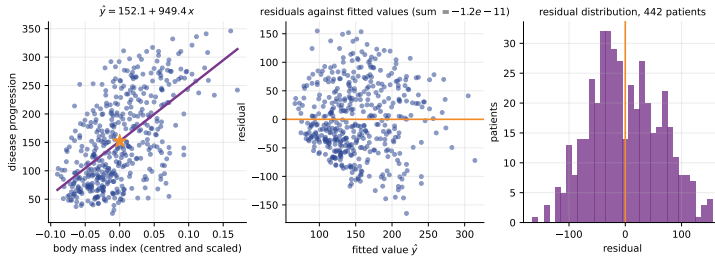

All eight steps, once, on real patients

```
1 problem : does body mass index predict disease progression?
2 variables : y = progression score; x = bmi (centred and scaled here)
3 data : 442 patients, 331 train / 111 test
4 model :  $y = b_0 + b_1 x + e$ 
5 method : least squares
6 fit : LinearRegression b0 = 153.2257 b1 = 1016.9235
      by hand b0 = 153.2257 b1 = 1016.9235
7 check : train RMSE 61.711 test RMSE 64.664
      training residuals sum to 2.274e-12
      5-fold RMSE 62.605 +/- 2.888
      always predicting ybar gives RMSE 77.006
8 use : one sd more bmi predicts 49.4 more progression points
```

Read line 7 twice. 62.6 **against** 77.0: one column of one measurement removes about a fifth of the error of guessing the mean, and the test RMSE is close to the training RMSE, so nothing is over-fitted. Read line 6 too — **the library and the hand formula agree to four decimals**, which is the whole point of the next twenty minutes.

Drawn: step 7, on the same fit

One simple linear regression, worked end to end on load_diabetes



Left: the fitted line passes through the star, which is $(\bar{x}, \bar{y})$ — we prove that in twenty minutes.

Middle: residuals against fitted values, summing to 10^{-11} ; we prove that too. Right: they are roughly symmetric about zero, which is the assumption Lecture 9 will lean on. **The spread does widen to the right, and that is a real defect of this model.**

The model, stated properly

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad i = 1, \dots, n$$

Four assumptions

1. **Linearity:** $\mathbb{E}[y \mid x] = \beta_0 + \beta_1 x$.
2. **Independence:** the ε_i are independent.
3. **Constant variance:** $\text{Var}(\varepsilon_i) = \sigma^2$, the same for every x .
4. **Zero mean:** $\mathbb{E}[\varepsilon_i] = 0$.

Two lines, and do not confuse them

β_0, β_1 describe the **population** and you will never see them.

b_0, b_1 are **estimates** computed from your n rows, and they change if you collect the data again.

Notation for the rest of the lecture

$\hat{y}_i = b_0 + b_1 x_i$ is the fitted value; $e_i = y_i - \hat{y}_i$ is the residual. ε_i is unobservable, e_i is on

Which line? We must choose a criterion

Candidate criteria

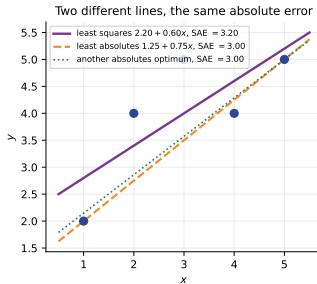
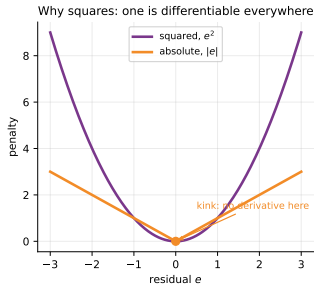
- minimise $\sum_i e_i$ — **useless**: positive and negative errors cancel, and infinitely many lines score zero.
- minimise $\sum_i |e_i|$ — sensible, robust, and has *no closed form*.
- minimise $\max_i |e_i|$ — lets one bad row dictate the whole line.
- minimise $\sum_i e_i^2$ — **least squares**.

Why squares win

1. Differentiable everywhere, so calculus applies.
2. The derivative is *linear* in b_0, b_1 , so the stationary condition is a linear system.
3. Convex, so the stationary point is the global minimum.
4. Under Normal noise it is also the maximum-likelihood answer — Lecture 9.

And one honest cost: squaring makes the criterion **sensitive to outliers**, which we measure later in the lecture.

Drawn: why squares, and what absolutes cost you



Left: $|e|$ has a corner at $e = 0$ and no derivative there, so “set the derivative to zero” is not available. Right: on our five points the two dashed lines have **exactly the same** total absolute error (3.00), so the least-absolute-deviation answer is *not unique*. Least squares has one answer, always.

The object we are minimising

For a given pair (b_0, b_1) the total squared error is

$$\text{SSE}(b_0, b_1) = \sum_{i=1}^n (y_i - b_0 - b_1 x_i)^2.$$

Two things to notice before differentiating

- x_i and y_i are **known numbers**. The unknowns are b_0 and b_1 — two of them, not n .
- Each term is a square, so $\text{SSE} \geq 0$: the surface is a bowl. It has a bottom, and we are about to find it exactly.

This is the L3 objective, up to a constant

$J(w) = \frac{1}{n} \|Xw - y\|^2 = \frac{1}{n} \text{SSE}$. Dividing by n does not move the minimiser. **Same function, same answer, two ways of getting there.**

Step 1: the two partial derivatives

Differentiate term by term, using the chain rule on $(y_i - b_0 - b_1 x_i)^2$:

$$\frac{\partial \text{SSE}}{\partial b_0} = \sum_{i=1}^n 2(y_i - b_0 - b_1 x_i) \cdot (-1) = -2 \sum_{i=1}^n (y_i - b_0 - b_1 x_i),$$

$$\frac{\partial \text{SSE}}{\partial b_1} = \sum_{i=1}^n 2(y_i - b_0 - b_1 x_i) \cdot (-x_i) = -2 \sum_{i=1}^n x_i (y_i - b_0 - b_1 x_i).$$

The bracket is exactly the residual e_i . So the two derivatives are $-2 \sum e_i$ and $-2 \sum x_i e_i$ — remember that; it gives us two properties of the fitted line for free.

Step 2: set both to zero — the normal equations

Dividing by -2 and expanding:

$$\begin{aligned}\sum_i y_i - nb_0 - b_1 \sum_i x_i &= 0 &\implies & n b_0 + b_1 \sum_i x_i = \sum_i y_i, \\ \sum_i x_i y_i - b_0 \sum_i x_i - b_1 \sum_i x_i^2 &= 0 &\implies & b_0 \sum_i x_i + b_1 \sum_i x_i^2 = \sum_i x_i y_i.\end{aligned}$$

Two linear equations in two unknowns

These are the **normal equations**. Every quantity in them — n , $\sum x_i$, $\sum y_i$, $\sum x_i^2$, $\sum x_i y_i$ — is a number you can read off the data in one pass.

Notice what has happened: a minimisation over a continuum of lines has become a **system of two linear equations**. That is the whole trick, and it is available because the derivative of a square is linear.

Step 3: solve them

Divide the first equation by n :

$$b_0 + b_1 \bar{x} = \bar{y} \implies \boxed{b_0 = \bar{y} - b_1 \bar{x}}$$

Substitute into the second and collect the b_1 terms:

$$(\bar{y} - b_1 \bar{x}) \sum_i x_i + b_1 \sum_i x_i^2 = \sum_i x_i y_i \implies b_1 \left(\sum_i x_i^2 - \bar{x} \sum_i x_i \right) = \sum_i x_i y_i - \bar{y} \sum_i x_i.$$

Using $\sum_i x_i = n\bar{x}$, the two brackets are $\sum x_i^2 - n\bar{x}^2$ and $\sum x_i y_i - n\bar{x}\bar{y}$, which are exactly

$$S_{xx} = \sum_i (x_i - \bar{x})^2, \quad S_{xy} = \sum_i (x_i - \bar{x})(y_i - \bar{y}),$$

$$b_1 = \frac{S_{xy}}{S_{xx}}, \quad b_0 = \bar{y} - b_1 \bar{x}$$

Why those two brackets really are S_{xx} and S_{xy}

This expansion is worth doing once, because it appears in every regression proof you will meet:

$$\sum_i (x_i - \bar{x})^2 = \sum_i x_i^2 - 2\bar{x} \sum_i x_i + n\bar{x}^2 = \sum_i x_i^2 - 2n\bar{x}^2 + n\bar{x}^2 = \sum_i x_i^2 - n\bar{x}^2,$$

$$\sum_i (x_i - \bar{x})(y_i - \bar{y}) = \sum_i x_i y_i - \bar{y} \sum_i x_i - \bar{x} \sum_i y_i + n\bar{x}\bar{y} = \sum_i x_i y_i - n\bar{x}\bar{y}.$$

Two forms of the same number

The left-hand forms are numerically safer; the right-hand forms are what you use in an examination, because $\sum x_i$, $\sum x_i^2$ and $\sum x_i y_i$ come straight out of a table of five rows.

Step 4: is it a minimum? The second-order condition

A stationary point could be a minimum, a maximum or a saddle. Take the second derivatives:

$$\frac{\partial^2 \text{SSE}}{\partial b_0^2} = 2n, \quad \frac{\partial^2 \text{SSE}}{\partial b_1^2} = 2 \sum_i x_i^2, \quad \frac{\partial^2 \text{SSE}}{\partial b_0 \partial b_1} = 2 \sum_i x_i.$$

$$H = 2 \begin{pmatrix} n & \sum_i x_i \\ \sum_i x_i & \sum_i x_i^2 \end{pmatrix}, \quad \det H = 4 \left(n \sum_i x_i^2 - (\sum_i x_i)^2 \right) = 4n S_{xx}.$$

$H_{11} = 2n > 0$ and $\det H = 4n S_{xx} > 0$ whenever the x_i are not all equal. **So H is positive definite, SSE is strictly convex, and the stationary point is the unique global minimum.**

And the exception is instructive: if every x_i is the same then $S_{xx} = 0$, the data is a vertical strip, and *no* line is determined. The algebra tells you so by dividing by zero.

The second-order condition, checked numerically

```
Xm = np.column_stack([np.ones(n), x])  
H = 2.0 * Xm.T @ Xm # Hessian of SSE(b0, b1)
```

```
Hessian of SSE =  
[[ 10.  30.]  
 [ 30. 110.]]  
d2SSE/db0^2 = 2n      = 10.0  
determinant   = 200.0  
and 4*n*Sxx   = 200.0  
eigenvalues   = [ 1.6905 118.3095]  
positive definite -> the stationary point is a strict minimum: True  
if every x is identical: Sxx = 0.0 and det(X'X) = 0.0 -> no unique fit
```

The second-order condition, checked numerically

```
Xm = np.column_stack([np.ones(n), x])  
H = 2.0 * Xm.T @ Xm # Hessian of SSE(b0, b1)
```

```
Hessian of SSE =  
[[ 10.  30.]  
 [ 30. 110.]]  
d2SSE/db0^2 = 2n      = 10.0  
determinant    = 200.0  
and 4*n*Sxx    = 200.0  
eigenvalues    = [ 1.6905 118.3095]  
positive definite -> the stationary point is a strict minimum: True  
if every x is identical: Sxx = 0.0 and det(X'X) = 0.0 -> no unique fit
```

$\det H = 200$, and $4nS_{xx} = 4 \cdot 5 \cdot 10 = 200$. **Both eigenvalues positive**, so the bowl curves upward in every direction. The last line is the degenerate case the algebra warned about, produced on purpose.

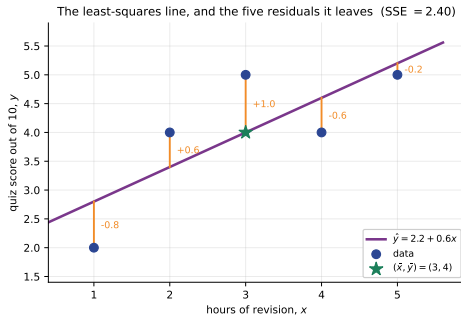
By hand: five students, hours against score

						Σ
x (hours of revision)	1	2	3	4	5	15
y (quiz score /10)	2	4	5	4	5	20
$x - \bar{x}$	-2	-1	0	1	2	0
$y - \bar{y}$	-2	0	1	0	1	0
$(x - \bar{x})^2$	4	1	0	1	4	10
$(x - \bar{x})(y - \bar{y})$	4	0	0	0	2	6

$$\bar{x} = 3, \quad \bar{y} = 4, \quad S_{xx} = 10, \quad S_{xy} = 6 \implies b_1 = \frac{6}{10} = 0.6, \quad b_0 = 4 - 0.6 \cdot 3 = 2.2.$$

$$\hat{y} = 2.2 + 0.6x \quad \text{—} \quad \text{an extra hour is worth 0.6 marks.}$$

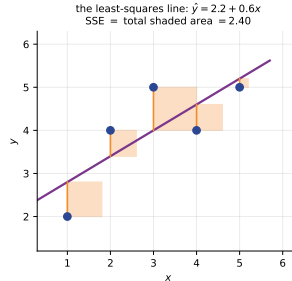
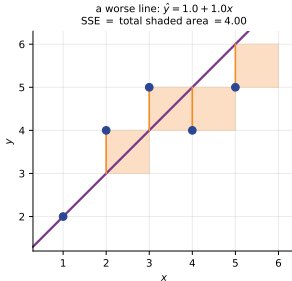
Drawn: the line and the five residuals



Orange bars are the residuals: $-0.8, +0.6, +1.0, -0.6, -0.2$. They sum to *exactly* zero, and the green star at $(3, 4)$ is on the line. Neither of those is a coincidence; both are forced by the two normal equations.

Drawn: least squares is minimising an area

Least squares minimises the total area of these squares



Each residual is drawn as a literal square of side $|e_i|$. The total shaded area is the SSE: 4.00 on the left, 2.40 on the right. **Least squares slides the line until that total area is as small as it can be.**

Predict first #1

Four candidate lines for the five points. **Rank them by SSE, and say which of them have residuals summing to zero.**

A $\hat{y} = 2.0 + 0.7x$

B $\hat{y} = 2.2 + 0.6x$

C $\hat{y} = 2.5 + 0.5x$

D $\hat{y} = 1.0 + 1.0x$

Predict first #1

Four candidate lines for the five points. **Rank them by SSE, and say which of them have residuals summing to zero.**

-
- A $\hat{y} = 2.0 + 0.7x$
 B $\hat{y} = 2.2 + 0.6x$
 C $\hat{y} = 2.5 + 0.5x$
 D $\hat{y} = 1.0 + 1.0x$
-

	SSE	$\sum e_i$	$\sum x_i e_i$
A: $2.0 + 0.7x$	2.5500	-0.5000	-2.5000
B: $2.2 + 0.6x$	2.4000	0.0000	0.0000
C: $2.5 + 0.5x$	2.5000	0.0000	+1.0000
D: $1.0 + 1.0x$	4.0000	0.0000	-4.0000

Predict first #1

Four candidate lines for the five points. **Rank them by SSE, and say which of them have residuals summing to zero.**

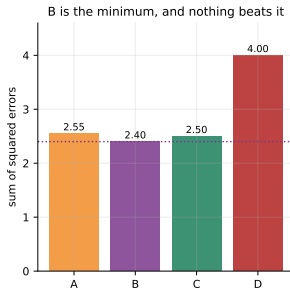
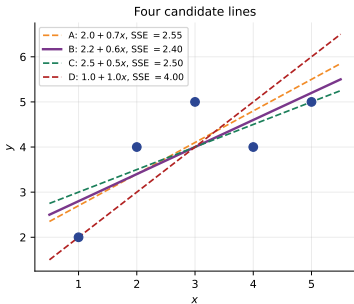
- | | |
|---|------------------------|
| A | $\hat{y} = 2.0 + 0.7x$ |
| B | $\hat{y} = 2.2 + 0.6x$ |
| C | $\hat{y} = 2.5 + 0.5x$ |
| D | $\hat{y} = 1.0 + 1.0x$ |

	SSE	$\sum e_i$	$\sum x_i e_i$
A: $2.0 + 0.7x$	2.5500	-0.5000	-2.5000
B: $2.2 + 0.6x$	2.4000	0.0000	0.0000
C: $2.5 + 0.5x$	2.5000	0.0000	+1.0000
D: $1.0 + 1.0x$	4.0000	0.0000	-4.0000

Three of the four have residuals summing to zero

$\sum e_i = 0$ is **necessary, not sufficient**. It is only *one* of the two normal equations. B is the only line that also satisfies $\sum x_i e_i = 0$, and B is the one with the smallest SSE.

Drawn: the four candidates and their scores



C is visually almost as good as B and scores 2.50 against 2.40. The difference is small because we are near the bottom of a bowl, where the surface is flat — which is also why gradient descent takes so long to finish the job.

Predict first #2

Same five numbers, now through the library. **To how many decimal places will np.polyfit and LinearRegression agree with the hand arithmetic?**

```
p = np.polyfit(x, y, 1) # returns [slope, intercept]
lr = LinearRegression().fit(x.reshape(-1, 1), y)
```

Predict first #2

Same five numbers, now through the library. **To how many decimal places will np.polyfit and LinearRegression agree with the hand arithmetic?**

```
p = np.polyfit(x, y, 1) # returns [slope, intercept]
lr = LinearRegression().fit(x.reshape(-1, 1), y)
```

```
by hand          b0 = 2.200000000000    b1 = 0.600000000000
np.polyfit        b0 = 2.200000000000    b1 = 0.600000000000
LinearRegression  b0 = 2.200000000000    b1 = 0.600000000000
max abs difference, hand vs polyfit          4.441e-16
max abs difference, hand vs LinearRegression 0.000e+00
```

Predict first #2

Same five numbers, now through the library. **To how many decimal places will np.polyfit and LinearRegression agree with the hand arithmetic?**

```
p = np.polyfit(x, y, 1) # returns [slope, intercept]
lr = LinearRegression().fit(x.reshape(-1, 1), y)
```

```
by hand          b0 = 2.200000000000    b1 = 0.600000000000
np.polyfit        b0 = 2.200000000000    b1 = 0.600000000000
LinearRegression  b0 = 2.200000000000    b1 = 0.600000000000
max abs difference, hand vs polyfit          4.441e-16
max abs difference, hand vs LinearRegression 0.000e+00
```

Not “close”. Identical, to machine precision

LinearRegression matched the hand arithmetic **exactly**; polyfit differs in the last bit of a double, because it goes through an SVD. **The library is not doing anything you did not just do on paper.** It is doing it faster, and with better numerics on larger problems.

Four properties, all free consequences of the derivation

From the first normal equation

$$\sum_i e_i = 0 \quad \text{and hence} \quad \bar{\hat{y}} = \bar{y}.$$

The residuals cancel exactly. **Only because an intercept was fitted:** that equation is the derivative with respect to b_0 .

$$b_0 = \bar{y} - b_1 \bar{x} \Rightarrow \hat{y}(\bar{x}) = \bar{y}.$$

The line passes through the centroid $(\bar{x}, \bar{y})$.

From the second

$$\sum_i x_i e_i = 0 \quad \text{and hence} \quad \sum_i \hat{y}_i e_i = 0.$$

The residual vector is orthogonal to the predictor *and* to the fitted values — there is no linear signal left in e .

And the consequence

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{Total SS}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{Explained SS}} + \underbrace{\sum_i e_i^2}_{\text{Residual SS}}$$

UPES
UNIVERSITY OF PRACTICE & EXPERIENCE

The properties, on the five points

x	y	yhat	residual
1	2.0	2.80	-0.80
2	4.0	3.40	0.60
3	5.0	4.00	1.00
4	4.0	4.60	-0.60
5	5.0	5.20	-0.20

sum of residuals -0.0000000000000000
sum of $x_i * e_i$ -0.0000000000000000
sum of $yhat_i * e_i$ -0.0000000000000001
mean of yhat = 4.0000 and ybar = 4.0000
the line at xbar = 3.0 gives 4.0000 and ybar = 4.0000

The properties, on the five points

x	y	yhat	residual
1	2.0	2.80	-0.80
2	4.0	3.40	0.60
3	5.0	4.00	1.00
4	4.0	4.60	-0.60
5	5.0	5.20	-0.20

sum of residuals = -0.0000000000000000
 sum of $x_i * e_i$ = -0.0000000000000000
 sum of $yhat_i * e_i$ = -0.0000000000000001
 mean of yhat = 4.0000 and $ybar = 4.0000$
 the line at $xbar = 3.0$ gives 4.0000 and $ybar = 4.0000$

SSE (residual) = 2.4000
 SSR (explained) = 3.6000
 SST (total) = 6.0000
 SSR + SSE = 6.0000 and SST = 6.0000
 check: $SSR = S_{xy}^2 / S_{xx} = 3.6000$
 correlation $r = S_{xy} / \sqrt{S_{xx} * S_{yy}} = 0.7746$
 and $b1 = r * sd(y) / sd(x) = 0.7746 * 1.2247 / 1.5811 = 0.6000$

Drop the intercept and watch the first property die

Fit $\hat{y} = b_1 x$ instead. There is now only *one* normal equation, the one from $\partial/\partial b_1$:

```
b1_noint = np.sum(x * y) / np.sum(x * x) # fit y = b1 x, no intercept
```

```
no-intercept fit  yhat = 1.2000 x
residuals        [ 0.8  1.6  1.4 -0.8 -1. ]
sum of residuals +2.0000    (NOT zero)
sum of x_i * e_i +1.7764e-15 (still zero -- that equation was solved)
SSE              6.8000    against 2.4000 with an intercept
```

Drop the intercept and watch the first property die

Fit $\hat{y} = b_1 x$ instead. There is now only *one* normal equation, the one from $\partial/\partial b_1$:

```
b1_noint = np.sum(x * y) / np.sum(x * x) # fit y = b1 x, no intercept
```

```
no-intercept fit  yhat = 1.2000 x
residuals        [ 0.8  1.6  1.4 -0.8 -1. ]
sum of residuals  +2.0000    (NOT zero)
sum of x_i * e_i  +1.7764e-15 (still zero -- that equation was solved)
SSE               6.8000    against 2.4000 with an intercept
```

The residuals now sum to +2, and the SSE nearly trebles

$\sum e_i = 0$ was never a law of nature. It is the $\partial SSE/\partial b_0 = 0$ equation, and if there is no b_0 there is no equation. **Whenever someone tells you residuals sum to zero, the words “with an intercept” are doing the work.**

How good is the slope? S_{xx} is the answer

Under the four assumptions one can show

$$\text{Var}(b_1) = \frac{\sigma^2}{S_{xx}}, \quad \text{Var}(b_0) = \sigma^2 \left(\frac{1}{n} + \frac{\bar{x}^2}{S_{xx}} \right).$$

Read the first formula

S_{xx} is in the denominator, so **the more spread out your x values are, the more precisely you learn the slope**. Doubling the spread of x halves the standard error of b_1 , for free, with the same number of rows.

The design lesson

If you can *choose* where to observe — an experiment, a calibration, a load test — put your points at the extremes of the usable range, not clustered in the middle. Same n , better estimate.

Predict first #3

20 points, noise sd 1.0, true slope 2.0. Refit 4000 times. **If I widen the range of x from ± 0.5 to ± 4 , what happens to the spread of the fitted slopes?**

Predict first #3

20 points, noise sd 1.0, true slope 2.0. Refit 4000 times. **If I widen the range of x from ± 0.5 to ± 4 , what happens to the spread of the fitted slopes?**

half-range of x	S_{xx}	sd of 4000 fitted slopes	$\sigma/\sqrt{S_{xx}}$
0.5	1.842	0.7237	0.7368
1.0	7.368	0.3683	0.3684
2.0	29.474	0.1859	0.1842
4.0	117.895	0.0932	0.0921

Predict first #3

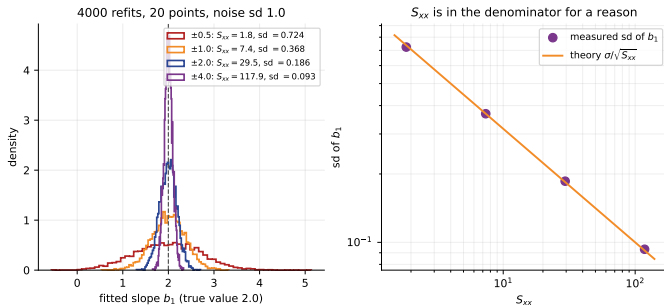
20 points, noise sd 1.0, true slope 2.0. Refit 4000 times. **If I widen the range of x from ± 0.5 to ± 4 , what happens to the spread of the fitted slopes?**

half-range of x	S_{xx}	sd of 4000 fitted slopes	sigma/sqrt(S_{xx})
0.5	1.842	0.7237	0.7368
1.0	7.368	0.3683	0.3684
2.0	29.474	0.1859	0.1842
4.0	117.895	0.0932	0.0921

A factor of 7.8, from the same 20 rows

Eight times the range of x , eight times the precision — because S_{xx} grew $64\times$ and the standard error goes as $1/\sqrt{S_{xx}}$. **The measured column and the theory column agree to three decimal places, so the formula is not decoration.**

Drawn: the same result, twice



Left: four sampling distributions of b_1 , all centred on the true value 2.0 — least squares is unbiased whatever the spread — but with very different widths. Right: the measured widths against $\sigma/\sqrt{S_{xx}}$ on log axes, and they sit on the line.

The same derivation, written once for any number of co

Stack the data: X is $n \times 2$ with a column of ones, y is $n \times 1$, $b = (b_0, b_1)^\top$.

$$\text{SSE}(b) = \|y - Xb\|^2 = (y - Xb)^\top (y - Xb) = y^\top y - 2b^\top X^\top y + b^\top X^\top X b.$$

Differentiate with respect to the vector b and set to zero:

$$\frac{\partial \text{SSE}}{\partial b} = -2X^\top y + 2X^\top X b = 0 \quad \implies \quad \boxed{X^\top X b = X^\top y}$$

If $X^\top X$ is invertible, $b = (X^\top X)^{-1} X^\top y$, and the Hessian is $2X^\top X$, positive definite exactly when X has full column rank.

Two scalar equations became one matrix equation, and **nothing in the line above mentions the number of columns**. That is Lecture 11, already done.

The matrix route on the five points, entirely by hand

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{pmatrix}, \quad X^T X = \begin{pmatrix} 5 & 15 \\ 15 & 55 \end{pmatrix}, \quad X^T y = \begin{pmatrix} 20 \\ 66 \end{pmatrix}$$

$$\det(X^T X) = 275 - 225 = 50 = n S_{xx}, \quad (X^T X)^{-1} = \frac{1}{50} \begin{pmatrix} 55 & -15 \\ -15 & 5 \end{pmatrix}$$

$$b = \frac{1}{50} \begin{pmatrix} 55 & -15 \\ -15 & 5 \end{pmatrix} \begin{pmatrix} 20 \\ 66 \end{pmatrix} = \frac{1}{50} \begin{pmatrix} 1100 - 990 \\ -300 + 330 \end{pmatrix} = \frac{1}{50} \begin{pmatrix} 110 \\ 30 \end{pmatrix} = \begin{pmatrix} 2.2 \\ 0.6 \end{pmatrix}$$

$\sum x_i = 15$, $\sum x_i^2 = 55$, $\sum y_i = 20$, $\sum x_i y_i = 2 + 8 + 15 + 16 + 25 = 66$. **Four sums, one 2×2 inverse, and the same answer as the scalar formulas.**

And in code, four ways

```
Xm = np.column_stack([np.ones(n), x])  
beta = np.linalg.inv(Xm.T @ Xm) @ (Xm.T @ y)
```

```
X'X =  
[[ 5. 15.]  
 [15. 55.]  
X'y = [20. 66.]  
det(X'X) = 50.0000    and n*Sxx = 50.0000  
inv(X'X) =  
[[ 1.1 -0.3]  
 [-0.3  0.1]]  
beta = inv(X'X) X'y = [2.2 0.6]  
np.linalg.solve      = [2.2 0.6]  
np.linalg.lstsq      = [2.2 0.6]  
np.linalg.pinv       = [2.2 0.6]  
agrees with the scalar formulas to 2.665e-15
```

And in code, four ways

```
Xm = np.column_stack([np.ones(n), x])
beta = np.linalg.inv(Xm.T @ Xm) @ (Xm.T @ y)
```

```
X'X =
[[ 5. 15.]
 [15. 55.]]
X'y = [20. 66.]
det(X'X) = 50.0000    and n*Sxx = 50.0000
inv(X'X) =
[[ 1.1 -0.3]
 [-0.3  0.1]]
beta = inv(X'X) X'y = [2.2 0.6]
np.linalg.solve      = [2.2 0.6]
np.linalg.lstsq      = [2.2 0.6]
np.linalg.pinv       = [2.2 0.6]
agrees with the scalar formulas to 2.665e-15
```

$1/50 \times 55 = 1.1$ and $1/50 \times (-15) = -0.3$: the printed inverse is the fraction you wrote on the board. **Use solve or lstsq in real code, never inv** — same answer here, better numerics when $X^T X$ is nearly singular.

Lecture 3, revisited with today's model

What L3 did

$$J(w) = \frac{1}{n} \|Xw - y\|^2, \quad \nabla J(w) = \frac{2}{n} X^T (Xw - y)$$

$$w \leftarrow w - \eta \nabla J(w)$$

Repeat until it stops moving. Choose η ; hope.

What L8 does

$$\nabla J(w) = 0 \implies X^T X w = X^T y$$

$$w = (X^T X)^{-1} X^T y$$

One linear solve. No step size, no epochs, no tolerance.

They are the same objective

Gradient descent *approaches* the point where the gradient is zero. Least squares *solves* for it. The only reason L3 iterated is that in general you cannot solve $\nabla J = 0$ in closed form — and for a linear model with squared error, you can.

Predict first #4

Gradient descent on SSE for the five points, from $(0,0)$, $\eta = 0.01$. **How many iterations before it matches $(2.2, 0.6)$ to four decimal places?**

Predict first #4

Gradient descent on SSE for the five points, from $(0,0)$, $\eta = 0.01$. **How many iterations before it matches $(2.2, 0.6)$ to four decimal places?**

iter	b0	b1	SSE	max theta-theta*
1	0.4000	1.3200	8.2320	1.80e+00
2	0.3640	1.0680	5.5234	1.84e+00
5	0.4655	1.0807	5.1381	1.73e+00
10	0.6072	1.0412	4.7089	1.59e+00
50	1.3946	0.8231	2.9903	8.05e-01
200	2.1376	0.6173	2.4035	6.24e-02
1000	2.2000	0.6000	2.4000	7.44e-08
4000	2.2000	0.6000	2.4000	1.24e-14
closed form		2.2000	0.6000	2.4000

gradient descent matched the closed form to 4 decimals at iteration 619

Predict first #4

Gradient descent on SSE for the five points, from $(0,0)$, $\eta = 0.01$. **How many iterations before it matches $(2.2, 0.6)$ to four decimal places?**

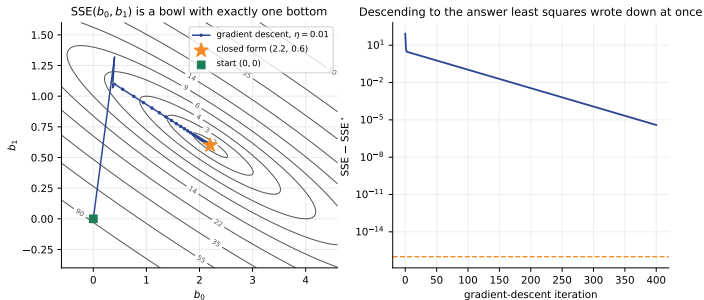
iter	b0	b1	SSE	max theta-theta*
1	0.4000	1.3200	8.2320	1.80e+00
2	0.3640	1.0680	5.5234	1.84e+00
5	0.4655	1.0807	5.1381	1.73e+00
10	0.6072	1.0412	4.7089	1.59e+00
50	1.3946	0.8231	2.9903	8.05e-01
200	2.1376	0.6173	2.4035	6.24e-02
1000	2.2000	0.6000	2.4000	7.44e-08
4000	2.2000	0.6000	2.4000	1.24e-14
closed form		2.2000	0.6000	2.4000

gradient descent matched the closed form to 4 decimals at iteration 619

619 iterations, to reproduce one division

And look at the SSE column: after 200 steps it is 2.4035 against the optimal 2.4000 — **the loss is essentially right long before the parameters are**. Near the bottom of a bowl the surface is flat, so a large parameter error costs almost no loss. That is why η -tuning felt so fiddly in L3.

Drawn: the bowl and the descent into it



Left: the SSE contours are ellipses, tilted because b_0 and b_1 are correlated (the $\sum x_i$ off-diagonal of $X^T X$), so descent zig-zags down a valley — the L6 ravine picture, again. Right: the gap to the optimal loss on a log scale. **The orange star was available from the start.**

And on the real data: full-batch, stochastic, exact

load_diabetes, bmi against progression, 442 patients. Full-batch gradient descent on standardised x , then the coefficients unwound to the original units:

full-batch GD	1 steps	b0 = 30.4267	b1 = 189.8871	J = 20008.2328
full-batch GD	10 steps	b0 = 135.7983	b1 = 847.4904	J = 4180.8086
full-batch GD	100 steps	b0 = 152.1335	b1 = 949.4353	J = 3890.4566
full-batch GD	2000 steps	b0 = 152.1335	b1 = 949.4353	J = 3890.4566
closed form		b0 = 152.1335	b1 = 949.4353	J = 3890.4566
difference in the slope		4.547e-13 (relative 4.441e-16)		
SGDRegressor (stochastic)		b0 = 149.5577	b1 = 891.1257	J = 3904.7835

And on the real data: full-batch, stochastic, exact

load_diabetes, bmi against progression, 442 patients. Full-batch gradient descent on standardised x , then the coefficients unwound to the original units:

full-batch GD	1 steps	b0 =	30.4267	b1 =	189.8871	J =	20008.2328
full-batch GD	10 steps	b0 =	135.7983	b1 =	847.4904	J =	4180.8086
full-batch GD	100 steps	b0 =	152.1335	b1 =	949.4353	J =	3890.4566
full-batch GD	2000 steps	b0 =	152.1335	b1 =	949.4353	J =	3890.4566
closed form		b0 =	152.1335	b1 =	949.4353	J =	3890.4566
difference in the slope		4.547e-13 (relative 4.441e-16)					
SGDRegressor (stochastic)		b0 =	149.5577	b1 =	891.1257	J =	3904.7835

Full-batch arrives exactly; stochastic stops 6.1% short

100 steps of full-batch descent reproduce $(X^T X)^{-1} X^T y$ to 13 decimal places. SGDRegressor with a constant step lands 6.1% away on the slope — **that is the L3 noise floor, not a bug**: a constant-step stochastic method converges to a *region*, not a point.

So when do you use which?

	Closed form	Gradient descent
Cost	$O(nd^2 + d^3)$, one pass	$O(nd)$ per step, many steps
Memory	needs $X^T X$, or all of X	one mini-batch at a time
Tuning	none	step size, schedule, batch size, stopping
Answer	exact	as close as you can afford
Fails when	d large, $X^T X$ singular	almost never, slowly
Extends to	squared loss on a linear model	anything differentiable

Rule of thumb

Small d (say under a few thousand columns) and the data fits in memory: **use the closed form**, which is what `LinearRegression` does. Huge n , huge d , streaming data, or a model that is not linear-with-squared-loss: **descend**.

Logistic regression, two lectures away, has *no* closed form. That is why L3 came first.

Predict first #5

Take the five points and move *one* y value from 5 to 9. **Does it matter which one?**
Move the point at $x = 3$, then the point at $x = 5$. Which change moves the slope more?

Predict first #5

Take the five points and move *one* y value from 5 to 9. **Does it matter which one? Move the point at $x = 3$, then the point at $x = 5$. Which change moves the slope more?**

point moved	b0	b1	change in b1
none	2.2000	0.6000	+0.0000
y at x=3: 5 -> 9	3.0000	0.6000	+0.0000
y at x=5: 5 -> 9	0.6000	1.4000	+0.8000
y at x=1: 2 -> -2	-1.0000	1.4000	+0.8000

leverage $(x - \bar{x})^2 / S_{xx}$: [0.4 0.1 0. 0.1 0.4]

Predict first #5

Take the five points and move *one* y value from 5 to 9. **Does it matter which one? Move the point at $x = 3$, then the point at $x = 5$. Which change moves the slope more?**

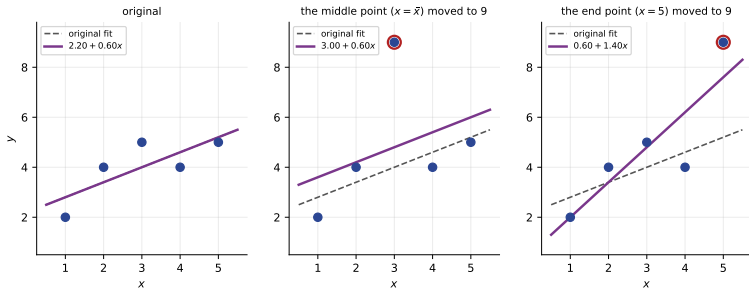
point moved	b0	b1	change in b1
none	2.2000	0.6000	+0.0000
y at x=3: 5 -> 9	3.0000	0.6000	+0.0000
y at x=5: 5 -> 9	0.6000	1.4000	+0.8000
y at x=1: 2 -> -2	-1.0000	1.4000	+0.8000
leverage $(x - \bar{x})^2 / S_{xx}$: [0.4 0.1 0. 0.1 0.4]			

The middle point cannot change the slope. At all

Its leverage is **exactly zero**, because $x_3 = \bar{x}$ and $b_1 = \sum (x_i - \bar{x})(y_i - \bar{y}) / S_{xx}$ weights each row by $(x_i - \bar{x})$. Move it anywhere you like: only b_0 responds. The two end points carry 0.4 of the leverage each — **four points out of five hold 80% of the influence**.

Drawn: same size of change, different consequence

Leverage $(x_i - \bar{x})^2 / S_{xx} = [0.4 \ 0.1 \ 0. \ 0.1 \ 0.4]$: a point at $\bar{x}$ cannot change the slope



The same +4 error in y . In the middle panel the line lifts and keeps its slope; in the right panel it pivots. **“Is this an outlier?” is the wrong question — ask where it sits in x .**

Predict first #6 — Anscombe's quartet

Four datasets, eleven points each. Their means, S_{xx} , S_{xy} , fitted lines and SSE are printed below. **Before you look at the next slide: how similar do you expect the four scatterplots to be?**

Predict first #6 — Anscombe's quartet

Four datasets, eleven points each. Their means, S_{xx} , S_{xy} , fitted lines and SSE are printed below. **Before you look at the next slide: how similar do you expect the four scatterplots to be?**

set	xbar	ybar	Sxx	Sxy	b0	b1	SSE
I	9.00	7.50	110.00	55.01	3.0001	0.5001	13.7627
II	9.00	7.50	110.00	55.00	3.0009	0.5000	13.7763
III	9.00	7.50	110.00	54.97	3.0025	0.4997	13.7562
IV	9.00	7.50	110.00	54.99	3.0017	0.4999	13.7425

Predict first #6 — Anscombe's quartet

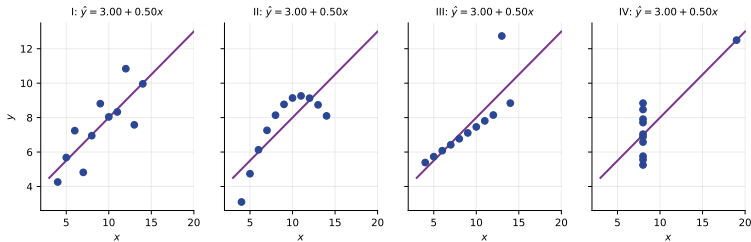
Four datasets, eleven points each. Their means, S_{xx} , S_{xy} , fitted lines and SSE are printed below. **Before you look at the next slide: how similar do you expect the four scatterplots to be?**

set	xbar	ybar	Sxx	Sxy	b0	b1	SSE
I	9.00	7.50	110.00	55.01	3.0001	0.5001	13.7627
II	9.00	7.50	110.00	55.00	3.0009	0.5000	13.7763
III	9.00	7.50	110.00	54.97	3.0025	0.4997	13.7562
IV	9.00	7.50	110.00	54.99	3.0017	0.4999	13.7425

Every column agrees to two decimal places. **Four datasets, indistinguishable by every number least squares computes.**

Drawn: and here they are

Anscombe's quartet: the same least-squares line, four times over



The estimates were never the point on their own

I is a reasonable linear fit. II is a **parabola** and the model is wrong. III is a perfect line plus **one** bad point. IV has ten points at $x = 8$ and one point at $x = 19$ that *alone* determines the slope — leverage = 1.0. **Always plot the data, and always plot the residuals.** That is Lecture 9.

One more thing least squares will not tell you

Every single-predictor regression on `load_diabetes`, ranked by SSE:

feature	b1	b0	SSE	corr(x,y)
bmi	949.435	152.133	1719581.8	0.5865
s5	916.137	152.133	1781701.4	0.5659
bp	714.738	152.133	2110158.3	0.4415
...				
sex	69.715	152.133	2616148.9	0.0431
total sum of squares about ybar: 2621009.1				

One more thing least squares will not tell you

Every single-predictor regression on `load_diabetes`, ranked by SSE:

feature	b1	b0	SSE	corr(x,y)
bmi	949.435	152.133	1719581.8	0.5865
s5	916.137	152.133	1781701.4	0.5659
bp	714.738	152.133	2110158.3	0.4415
...				
sex	69.715	152.133	2616148.9	0.0431
total sum of squares about ybar:			2621009.1	

Every intercept is 152.133, because $b_0 = \bar{y} - b_1\bar{x}$ and every column of this dataset is already centred, so $\bar{x} = 0$. The best single predictor removes 34% of the total sum of squares; the worst removes 0.2%.

One more thing least squares will not tell you

Every single-predictor regression on `load_diabetes`, ranked by SSE:

feature	b1	b0	SSE	corr(x,y)
bmi	949.435	152.133	1719581.8	0.5865
s5	916.137	152.133	1781701.4	0.5659
bp	714.738	152.133	2110158.3	0.4415
...				
sex	69.715	152.133	2616148.9	0.0431
total sum of squares about ybar:			2621009.1	

Every intercept is 152.133, because $b_0 = \bar{y} - b_1\bar{x}$ and every column of this dataset is already centred, so $\bar{x} = 0$. The best single predictor removes 34% of the total sum of squares; the worst removes 0.2%.

And none of this is causation. `bmi` predicting progression is a statement about *this table*, not about what happens if you change a patient's BMI.

Summary — twelve lines

1. Regression models a quantitative y as a systematic part plus noise: $y = f(x) + \varepsilon$. Two jobs — prediction and explanation — and you must say which.
2. A model is **linear** when it is linear in the *parameters*. A parabola is a linear model; $\beta_0 + \beta_0\beta_1x$ is not.
3. Eight steps: problem, variables, data, specification, method, fit, validation, use. Steps 1–3 decide the analysis.
4. Least squares minimises $SSE(b_0, b_1) = \sum (y_i - b_0 - b_1x_i)^2$. Squares because the criterion is differentiable and its derivative is *linear* in the unknowns.
5. Setting both partials to zero gives the **normal equations**, and solving them gives $b_1 = S_{xy}/S_{xx}$, $b_0 = \bar{y} - b_1\bar{x}$.
6. The Hessian is $2 \begin{pmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix}$ with $\det = 4nS_{xx} > 0$: a strict, unique, global minimum.
7. Five points, $S_{xx} = 10$, $S_{xy} = 6$: $\hat{y} = 2.2 + 0.6x$, $SSE = 2.40$. `np.polyfit` agrees to 4×10^{-16} ; `LinearRegression` to 0.
8. Matrix form: $X^T X b = X^T y$, and by hand $\frac{1}{50} \begin{pmatrix} 55 & -15 \\ -15 & 5 \end{pmatrix} \begin{pmatrix} 20 \\ 66 \end{pmatrix} = (2.2, 0.6)$.
9. With an intercept, $\sum e_i = 0$ and the line passes through $(\bar{x}, \bar{y})$. Drop the intercept and $\sum e_i$ became $+2.0$. $\sum e_i = 0$ is necessary but **not sufficient**: three of four candidate lines had it.
10. Gradient descent on the same objective took **619 iterations** to match one division, and on diabetes reproduced $(X^T X)^{-1} X^T y$ to 4×10^{-16} relative. Stochastic descent stopped 6.1% short.
11. $\text{Var}(b_1) = \sigma^2/S_{xx}$: widening the range of x eightfold cut the spread of the slope by $7.8\times$, on the same 20 rows.
12. **Anscombe's quartet**: four datasets with the same $\bar{x}, \bar{y}, S_{xx}, S_{xy}, b_0, b_1$ and SSE, and four completely different pictures.

Before the next lecture

Read

notes.pdf — the derivation written out with every step, the matrix version, the proof that the cross term vanishes, and **10 practice questions with full worked solutions**.

Do

Questions 1, 2 and 7. Questions 1 and 2 are by-hand least-squares computations of exactly the kind the test will ask; question 7 is the derivation, from scratch, on a blank page.

Next: maximum likelihood and adequacy

Where does squared error *come from*?

Assume Normal noise, write the likelihood, maximise it — and get today's estimator back. Then R^2 , residual diagnostics and over-fitting.

Carry one number across

$SSR/SST = 3.6/6.0 = 0.6$ on the five points, and $r^2 = 0.7746^2 = 0.6$ as well.

That is not a coincidence, and Lecture 9 gives it a name.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.